

LABORATORY OBSERVATION / RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: Kottana Roshini

Roll No.A24126552263

1. TITLE

Student Profile Manager using JavaScript Class and DOM Manipulation

2. AIM

To develop a web application using JavaScript that creates a Student object using a class, accepts student details as user input, and dynamically displays the student profile on the webpage using DOM manipulation when a button is clicked.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of classes and objects in JavaScript.
- To create a class with multiple properties.
- To create an object of the class using user-provided values.
- To select and manipulate elements of the DOM.
- To dynamically create HTML elements using JavaScript.
- To modify element content and append elements to the webpage.
- To handle button click events.
- To apply basic CSS styling to dynamically generated content.

4. THEORY

A class in JavaScript is a blueprint for creating objects with predefined properties and behaviour. The Document Object Model (DOM) represents an HTML page as a tree of objects that JavaScript can access and modify.

In this application, a Student class is defined with four properties: name, roll number, department and CGPA. When the user enters these details in an HTML form and clicks the "Generate Profile" button, a click event is triggered.

Inside the event handler, the values entered by the user are read using `document.getElementById().value`, and a new Student object is created from these values. New HTML elements (a container div, a heading and paragraphs) are then created using `document.createElement()`, their text content is set using `textContent`, and they are attached to the page using `appendChild()`.

The basic flow of the application is:

User Input (Form) → Button Click Event → Student Object Created → DOM Elements Created → Profile Displayed on Webpage

5. ALGORITHM / PROCEDURE

- Create the project folder with `index.html`, `style.css` and `script.js`.
- Link the CSS and JavaScript files to the HTML page.
- Create an HTML form with input fields for Name, Roll Number, Department and CGPA.
- Add a “Generate Profile” button below the form.
- In JavaScript, define a Student class with a constructor that accepts name, rollNumber, department and cgpa.
- Select the button and the profile container using `document.getElementById()`.
- Add a click event listener to the button.
- Inside the event listener, read the input values entered by the user.
- Create a new Student object using the values read from the form.
- Dynamically create a div, a heading and paragraph elements using `document.createElement()`.
- Set the text content of each element using the properties of the Student object.
- Append the newly created elements to the profile container using `appendChild()`.
- Apply CSS styling to the form and the dynamically generated profile card.
- Open `index.html` in the browser, enter details and test the button.
- Perform multiple test cases and verify the actual output with the expected output.

6. PROGRAM

index.html

<!DOCTYPE html>

```
<html lang="en">

<head>

<meta charset="UTF-8">

<title>Student Profile Manager</title>

<link rel="stylesheet" href="style.css">

</head>

<body>


    <h1>Student Profile Manager</h1>


    <div class="form-container">

        <label for="name">Name:</label>

        <input type="text" id="name" placeholder="Enter full name">


        <label for="rollno">Roll Number:</label>

        <input type="text" id="rollno" placeholder="Enter roll number">


        <label for="department">Department:</label>

        <input type="text" id="department" placeholder="Enter department">


        <label for="cgpa">CGPA:</label>

        <input type="number" step="0.01" id="cgpa" placeholder="Enter CGPA">


        <button id="generateBtn">Generate Profile</button>

    </div>


    <div id="profileContainer"></div>
```

```
<script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
    font-family: Arial, sans-serif;
    background-color: #f4f6f8;
    text-align: center;
    padding: 30px;
}

.form-container {
    display: inline-block;
    text-align: left;
    background: #ffffff;
    padding: 20px 25px;
    border-radius: 8px;
    box-shadow: 0 2px 6px rgba(0, 0, 0, 0.15);
}

.form-container label {
    display: block;
    margin-top: 10px;
    font-weight: bold;
}

.form-container input {
    width: 250px;
```

```
padding: 6px 8px;
margin-top: 4px;
border: 1px solid #ccc;
border-radius: 4px;
}
```

```
#generateBtn {
  margin-top: 18px;
  padding: 8px 18px;
  background-color: #2e5496;
  color: #ffffff;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
```

```
.profile-card {
  display: inline-block;
  text-align: left;
  margin-top: 25px;
  padding: 20px 30px;
  background: #ffffff;
  border-left: 5px solid #2e5496;
  border-radius: 8px;
  box-shadow: 0 2px 6px rgba(0, 0, 0, 0.15);
}
```

```
.profile-card h2 {
```

```
margin-top: 0;
color: #2e5496;
}
```

```
.profile-card p {
margin: 6px 0;
font-family: "Courier New", monospace;
}
```

script.js

// Student class with four properties

```
class Student {
  constructor(name, rollNumber, department, cgpa) {
    this.name = name;
    this.rollNumber = rollNumber;
    this.department = department;
    this.cgpa = cgpa;
  }
}
```

// DOM selection

```
const generateBtn = document.getElementById("generateBtn");
const profileContainer = document.getElementById("profileContainer");
```

// Event handling

```
generateBtn.addEventListener("click", () => {
```

// Read user-provided values

```
const name = document.getElementById("name").value;
```

```
const rollno = document.getElementById("rollno").value;  
const department = document.getElementById("department").value;  
const cgpa = document.getElementById("cgpa").value;
```

```
// Create a Student object from user input
```

```
const student = new Student(name, rollno, department, cgpa);
```

```
// Clear any previously generated profile
```

```
profileContainer.innerHTML = "";
```

```
// Dynamically create elements for the profile card
```

```
const card = document.createElement("div");
```

```
card.className = "profile-card";
```

```
const heading = document.createElement("h2");
```

```
heading.textContent = "Student Profile";
```

```
card.appendChild(heading);
```

```
const nameP = document.createElement("p");
```

```
nameP.textContent = "Name : " + student.name;
```

```
card.appendChild(nameP);
```

```
const rollP = document.createElement("p");
```

```
rollP.textContent = "Roll No : " + student.rollNumber;
```

```
card.appendChild(rollP);
```

```
const deptP = document.createElement("p");
```

```
deptP.textContent = "Department : " + student.department;
```

```

card.appendChild(deptP);

const cgpaP = document.createElement("p");
cgpaP.textContent = "CGPA    : " + student.cgpa;
card.appendChild(cgpaP);

// Add the new profile card to the webpage
profileContainer.appendChild(card);
});

```

7. CODE EXPLANATION

Code	Explanation
class Student { ... }	Defines a blueprint for a student object with four properties.
constructor(name, rollNumber, department, cgpa)	Initialises a new Student object with the values passed to it.
document.getElementById()	Selects an existing element from the DOM.
addEventListener("click", ...)	Runs a function whenever the button is clicked.
.value	Reads the current text entered inside an input field.
new Student(...)	Creates an object of the Student class using the form values.
document.createElement()	Dynamically creates a new HTML element.
.textContent	Sets the visible text inside a created element.
.appendChild()	Adds a newly created element to the webpage.
.innerHTML = ""	Clears the previous profile before displaying a new one.

8. TEST CASES AND EXECUTION DATA

Test Case	Name	Roll No.	Department	CGPA	Expected Result	Status
TC-01	Ravi Kumar	A23126552001	CSE (AI & ML)	8.42	Profile card displayed correctly	Pass
TC-02	Sneha Reddy	A23126552014	CSE (Data Science)	9.10	Profile card displayed correctly	Pass
TC-03	Mohammed Ali	A23126552027	CSE (Cyber Security)	7.65	Profile card displayed correctly	Pass
TC-04	Divya Sharma	A23126552035	CSE	0	Profile card displayed correctly	Pass

Additional Validation Test

Test Case	Input	Expected Result	Status
TC-05	Name field left empty and button clicked	Profile card is generated with the Name field shown as blank	Pass

9. OUTPUT

Output 1: Student Profile Manager – initial form

file:///D:/FullStack/StudentProfileManager/index.html

Student Profile Manager

Name:

Roll Number:

Department:

CGPA:

Generate Profile

Output 2: After entering details and clicking "Generate Profile"

file:///D:/FullStack/StudentProfileManager/index.html

Student Profile Manager

Name:
Ravi Kumar

Roll Number:
A23126552001

Department:
CSE (AI & ML)

CGPA:
8.42

Generate Profile

Student Profile

Name : Ravi Kumar
Roll No : A23126552001
Department : CSE (AI & ML)
CGPA : 8.42

10. RESULT

Thus, a Student Profile Manager application was successfully developed using JavaScript classes and DOM manipulation. A Student object was created using user-provided values, DOM elements were dynamically created and styled, and the student profile was displayed correctly on the webpage. The application was verified using multiple test cases.